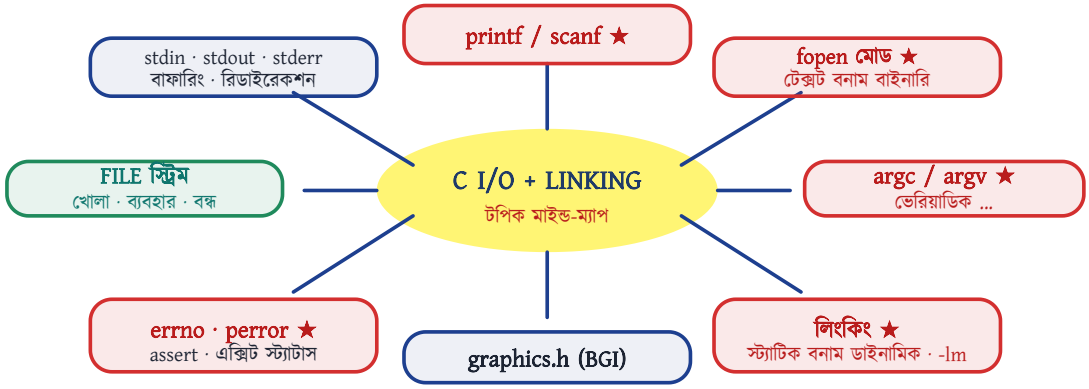
C-তে I/O, ফাইল, এরর ও লিংকিং

printf/scanf · fopen · fseek · stdarg · argc/argv · errno · লিংকিং · ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো লেসন এক পাতায়: লাল = বেশি আসে, সবুজ = যে ধারণার ওপর বাকি সব দাঁড়িয়ে

সিলেবাসের যে অংশ

স্ট্রাকচার্ড প্রোগ্রামিং সিলেবাসে আছে "**Input and Output: standard input and output, formatted input and output, file access; Variable length argument list; Command line parameters; Error Handling; Graphics; Linking; Library functions**"। ডেটা টাইপ, ফাংশন, পয়েন্টার ও স্ট্রাকচার আলাদা লেসনে।

হাই-ইল্ড তালিকা

① scanf: **%f** = float, **%lf** = double
② size_t (strlen, sizeof) → **%zu**
③ fopen ব্যর্থ ⇒ ফেরত দেয় **NULL**
④ "w" মুছে ফেলে · "a" সবসময় শেষে লেখে
⑤ **./prog one two** ⇒ argc = 3, argv[3] = NULL
⑥ কখনো va_arg(ap, float) নয় — double
⑦ perror লেখে **stderr**-এ
⑧ স্ট্যাটিক লিংক কেবল **দরকারি মডিউল** কপি করে; -lm = libm

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

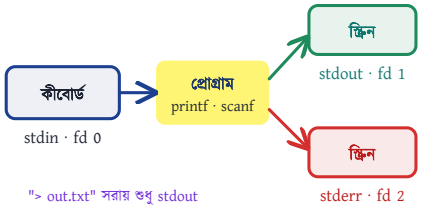
সূচিপত্র

✓ স্ট্রিম ও printf ... পৃ. ২
✓ scanf ও ক্যারেক্টার I/O ... পৃ. ৩
✓ ফাইল অ্যাক্সেস ও মোড ... পৃ. ৪
✓ র‍্যান্ডম অ্যাক্সেস ও EOF ... পৃ. ৫
✓ ভেরিয়াডিক ফাংশন ও argc/argv ... পৃ. ৬
✓ এরর হ্যান্ডলিং ... পৃ. ৭
✓ গ্রাফিক্স ও লাইব্রেরি ফাংশন ... পৃ. ৮
✓ কম্পাইলিং ও লিংকিং ... পৃ. ৯ · রাতের পাতা পৃ. ১০ · অনুশীলন পৃ. ১১

এই পুরো লেসনের উদ্দেশ্য: একটি C প্রোগ্রাম বাইরের জগতের সঙ্গে কথা বলে কেবল **স্ট্রিম** আর লাইব্রেরি কলের **রিটার্ন মান** দিয়ে, এবং চালানোর যোগ্য হয় কেবল তখনই যখন **লিংকার** একে ওই লাইব্রেরিগুলোর সঙ্গে জোড়ে — তাই পুরো লেসনটি হলো **স্ট্রিমে ঢোকো, যাচাই রাখো, শেষে লিংকার**।

১ স্ট্যান্ডার্ড স্ট্রিম ও ফরম্যাটেড আউটপুট (printf)

সংজ্ঞা: **স্ট্রিম (stream)** হলো বাইটের যেকোনো উৎস বা গন্তব্য — কীবোর্ড, স্ক্রিন, ফাইল, পাইপ — দেখার C-র একক উপায়, যা একটি `FILE *` দিয়ে ধরা হয়। প্রতিটি হোস্টেড C প্রোগ্রাম শুরু হয় **আগে থেকেই খোলা তিনটি স্ট্রিম** নিয়ে, যা `<stdio.h>`-এ ঘোষিত: **stdin** (কীবোর্ড), **stdout** (স্ক্রিন; টার্মিনালে লাইন-বাফারড, ফাইলে পুরো বাফারড) এবং **stderr** (স্ক্রিন, **আনবাফারড** — সঙ্গে সঙ্গে দেখায়)।



স্ক্রিনে একই রকম দেখায় — কিন্তু এরা **আলাদা স্ট্রিম**

পরীক্ষার ফাঁদ: (১) **stderr** কোনো লগ ফাইল নয়: এটিও স্ক্রিনে যায়, কিন্তু `prog > out.txt` কেবল stdout সরায়, তাই এরর তখনো দেখা যায়। (২) stdout-এর বাফারে থাকা লেখা **প্রোগ্রাম ক্র্যাশ করলে হারিয়ে যায়** — `fflush(stdout)` জোর করে বের করে দেয়।

স্পেসি.	আর্গুমেন্ট	ছাপে
%d %i	int	-42
%u	unsigned int	42
%ld %lld	long, long long	123456789
%f	**double (float উন্নীত হয়)**	3.140000
%Lf	long double	3.140000
%e %g	double: সূচক / ছোটটি	3.14e+00 · 3.14

স্পেসি.	আর্গুমেন্ট	ছাপে
%c	char (int হিসেবে)	A
%s	char * স্ট্রিং	Dhaka
%p	void * পয়েন্টার	0x7ffd...
%x %X %o	unsigned: হেক্স / অক্টাল	ff · FF · 377
%zu	**size_t**	5
%%	কিছু না	একটি % চিহ্ন

গঠন: % [ফ্ল্যাগ] [প্রস্থ] [.প্রিসিশন] [দৈর্ঘ্য] অক্ষর:

- বাঁয়ে সাজায় · **+** চিহ্ন দেখায় · **0** শূন্য দিয়ে ভরে · **#** %x-এ 0x, %o-তে 0

প্রস্থ (width) = **ন্যূনতম**, কখনো কাটে না; * দিলে আর্গুমেন্ট থেকে নেয়

.প্রিসিশন: দশমিকের পরের অঙ্ক (%f) · সর্বোচ্চ অক্ষর (%s)

দৈর্ঘ্য: hh, h, l, ll, L, z — আর্গুমেন্টের আকার

printf ফেরত দেয় ছাপা অক্ষরের সংখ্যা (এররে ঋণাত্মক): `printf("%d", printf("Hello"))` ছাপে **Hello5**।

সমাধানকৃত উদাহরণ ✍ আউটপুট বলো (_ = একটি স্পেস): `"%5d|"`, 42 → **___42|** · `"%-5d|"`, 42 → **42___|** · `"%05d"`, 42 → **00042** · `"%+d"`, 42 → **+42** · `"%.2f"`, 3.14159 → **3.14** · `"%8.3f|"`, 3.14159 → **___3.142|** · `"%#x %X %o"`, 255 ×3 → **0xff FF 377** · `"%.3s"`, "Bangladesh" → **Ban** · `"%2d"`, 12345 → **12345**।

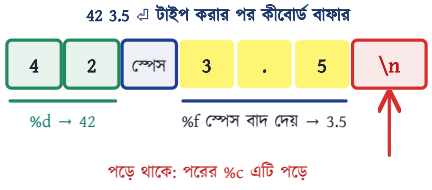
পরীক্ষার ফাঁদ: (১) **printf**-এ %f দিয়েই float ও double দুটোই চলে — float আর্গুমেন্ট double-এ উন্নীত হয়; পার্থক্যটা কেবল scanf-এ। (২) `printf("%d", 3.5)` **অনির্ধারিত আচরণ (undefined)**। (৩) `strlen` ফেরত দেয় **size_t**: ছাপো **%zu** দিয়ে, %d বা %lu নয়।

মনে রাখার কৌশল: "**ফ্ল্যাগ, প্রস্থ, ডট, দৈর্ঘ্য, অক্ষর**" — সবসময় এই ক্রমে। "**মাইনাস বাঁয়ে সরায়, শূন্য ভরে, প্লাস দেখায়।**"

২ ফরম্যাটেড ইনপুট (scanf) ও ক্যারেক্টার / লাইন I/O

সংজ্ঞা: scanf stdin থেকে অক্ষর পড়ে, ফরম্যাট অনুযায়ী রূপান্তর করে এবং **প্রতিটি ফল একটি পয়েন্টারের মাধ্যমে জমা রাখে** — তাই অ্যারে নয় এমন প্রতিটি আর্গুমেন্টে **&** লাগে। এটি ফেরত দেয় **যতগুলো আইটেম বসানো হলো সেই সংখ্যা**, অথবা প্রথম রূপান্তরের আগেই ইনপুট ব্যর্থ হলে **EOF**। %d, %f ও %s শুরুর ফাঁকা জায়গা (white space) বাদ দেয়; **%c দেয় না**।

টাইপ	printf	scanf	পার্থক্য কেন
float	%f	%f	পায় একটি **float *** — 4 বাইট
double	%f (C99 থেকে %lf-ও)	**%lf**	**double ***: 8 বাইট; %f ভরে কেবল 4
long double	%Lf	%Lf	বড় হাতের L, ছোট হাতের নয়
short / long	%hd / %ld	%hd / %ld	আকার পয়েন্টারের সঙ্গে মিলতে হবে
char	%c	%c, বা **" %c"**	স্পেসটি আগের '\n' বাদ দেয়
স্ট্রিং	%s	%s, **& ছাড়া**	অ্যারের নামই ঠিকানা; স্পেসে থামে



`scanf("%d %f", &n, &x)` এন্টার কী-র আগেই থামে

ফাংশন	কাজ
getchar() / putchar(c)	একটি অক্ষর ইন / আউট; getchar ফেরত দেয় **int**, যাতে EOF আঁটে
gets(s)	C11-এ বাতিল — আকারের সীমা নেই, বাফার ওভারফ্লো
fgets(s, n, stdin)	সর্বোচ্চ **n − 1** অক্ষর, **'\n' রেখে দেয়**, '\0' যোগ করে
puts(s) / fputs(s, fp)	puts '\n' যোগ করে; fputs কিছুই না
getch() / getche()	এন্টার ছাড়া একটি কী — **conio.h**, শুধু DOS/Turbo C, স্ট্যান্ডার্ড নয়

সমাধানকৃত উদাহরণ ✍ scanf কী ফেরত দেয়?: `int a = 0, b = 0, r = scanf("%d %d", &a, &b);` **(ক)** ইনপুট **10 20** → r = **2**, a = 10, b = 20। **(খ)** ইনপুট **10 abc** → r = **1**, b থাকে 0 এবং "abc" পরের পড়ার জন্য **বাফারেই থেকে যায়**। **(গ)** ইনপুট **abc** → r = **0**। **(ঘ)** ইনপুট শেষ (Ctrl+Z / Ctrl+D) → r = **EOF**। নিরাপদ লুপ: `while (scanf("%d", &x) == 1) sum += x;`

scanf-এর তিনটি কাজের কৌশল:

%9s — প্রস্থের সীমা: `char s[10]`-এ সর্বোচ্চ 9 অক্ষর

%[^\n] — স্ক্যানসেট: স্পেসসহ পুরো লাইন

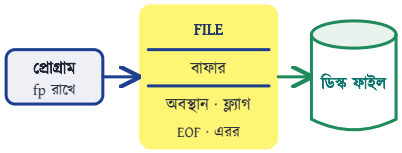
%*d — পড়ে **ফেলে দেয়**; রিটার্নে গোনা হয় না

পরীক্ষার ফাঁদ: (১) & ছাড়া `scanf("%d", x)` একটি মানকে ঠিকানা হিসেবে পাঠায় — অনির্ধারিত আচরণ, সাধারণত ক্র্যাশ। (২) যেখানে char আনসাইনড, সেখানে `char c = getchar();` কখনো EOF-এর সমান হয় না — **int** রাখো। (৩) স্ট্যান্ডার্ড C-তে `fflush(stdin)` অনির্ধারিত। (৪) %s কেবল **একটি শব্দ** পড়ে।

মনে রাখার কৌশল: "**printf নেয় মান, scanf নেয় ঠিকানা।**" আর "**lf = long float = double**": scanf-এ float আর double-এর মাঝে ফারাক শুধু ওই l।

৩ ফাইল অ্যাক্সেস: FILE *, fopen মোড, টেক্সট বনাম বাইনারি

সংজ্ঞা: ফাইল ব্যবহার করা হয় একটি **FILE স্ট্রাকচারের পয়েন্টার** দিয়ে (<stdio.h>-এ ঘোষিত), যেটি রাখে **বাফার**, **বর্তমান অবস্থান (position)** এবং **ফাইল-শেষ ও এরর ফ্ল্যাগ**। প্রতিটি ফাইলের কাজের তিন ধাপ: `fopen(name, mode)` দিয়ে **খোলা**, যা **ব্যর্থ হলে NULL ফেরত দেয়**; **প্রসেস** করা; `fclose(fp)` দিয়ে **বন্ধ**, যা বাফার খালি করে 0 ফেরত দেয় (ব্যর্থ হলে EOF)।



fp = fopen(...) — NULL কিনা যাচাই করো
fclose(fp) বাফার ডিস্কে লিখে দেয়

প্রোগ্রাম রাখে শুধু একটি পয়েন্টার; হিসাব রাখে FILE

মোড	ফাইল থাকলে	না থাকলে	অনুমতি
"r"	শুরু থেকে খোলে	NULL	পড়া
"w"	**শূন্যে কেটে ফেলে**	তৈরি করে	লেখা
"a"	রাখে; **প্রতিটি** লেখা যায় শেষে	তৈরি করে	যোগ (append)
"r+"	খোলে, কাটে **না**	NULL	পড়া + লেখা
"w+"	কেটে ফেলে	তৈরি করে	লেখা + পড়া
"a+"	রাখে; যেকোনো জায়গা পড়ে, শেষে লেখে	তৈরি করে	পড়া + যোগ

বাইনারি-র জন্য **b** যোগ করো: "rb", "wb", "ab", "r+b" (= "rb+"), "w+b", "a+b"। C11 যোগ করেছে **x** ("wx"): ওভাররাইট না করে ব্যর্থ হয়। মনে রাখো `printf(…) ≡ fprintf(stdout, …)`।

স্তর	পড়া	লেখা	শেষে / এররে সংকেত
অক্ষর	fgetc(fp), getc(fp)	fputc(c, fp), putc(c, fp)	EOF
লাইন / স্ট্রিং	fgets(s, n, fp)	fputs(s, fp)	NULL · EOF
ফরম্যাটেড	fscanf(fp, fmt, ...)	fprintf(fp, fmt, ...)	সংখ্যা বা EOF · ঋণাত্মক
ব্লক (বাইনারি) ★	fread(ptr, size, n, fp)	fwrite(ptr, size, n, fp)	যতগুলো হলো; শেষে বা এররে **< n**

```
/* copy in.txt to out.txt */
FILE *in  = fopen("in.txt", "r");
FILE *out = fopen("out.txt", "w");
if (in == NULL || out == NULL) {
    perror("fopen");
    return 1;
}
int c;              /* int, not char */
while ((c = fgetc(in)) != EOF)
    fputc(c, out);
fclose(in);
fclose(out);
```

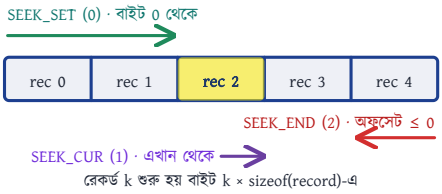
টেক্সট বনাম বাইনারি: **টেক্সট মোডে** লাইব্রেরি লাইনের শেষ বদলাতে পারে — Windows-এ '\n' লেখা হয় "\r\n" হিসেবে আর পড়ার সময় আবার '\n'। **বাইনারি মোড** বাইট হুবহু সরায়। Linux/UNIX-এ দুই মোড একই। `fprintf(fp, "%d", 12345)` লেখে **5টি অক্ষর**; `fwrite(&n, sizeof n, 1, fp)` লেখে **sizeof(int) বাইট** (বেশিরভাগ সিস্টেমে 4)।

পরীক্ষার ফাঁদ: (১) **"w" পুরোনো লেখা মুছে দেয়**; রাখতে চাইলে "a"। (২) **"r+"** কখনো ফাইল তৈরি করে না। (৩) fopen ব্যর্থতা জানায় **NULL** দিয়ে, EOF বা -1 নয়। (৪) fclose ভুলে গেলে **বাফারের ডেটা হারাতে পারে**। (৫) "a"/"a+"-এ fseek-এর পরেও লেখা যায় শেষে।

মনে রাখার কৌশল: "**r পড়ে, w মোছে, a জোড়ে — আর + দেয় বাকি অর্ধেক।**" "**b মানে বাইট।**" ফাইল না থাকলে ব্যর্থ হয় কেবল দুটি r-মোড; সব w- ও a-মোড ফাইল তৈরি করে।

8 র‍্যান্ডম অ্যাক্সেস: fseek, ftell, rewind ও EOF-এর নিয়ম

সংজ্ঞা: প্রতিটি খোলা স্ট্রিমের একটি **ফাইল পজিশন ইন্ডিকেটর** থাকে — যে বাইট অফসেটে পরের পড়া বা লেখা হবে। ধারাবাহিক (sequential) I/O একে নিজে নিজে সামনে নেয়; **র‍্যান্ডম অ্যাক্সেস** একে সরাসরি সরায় `fseek(fp, offset, whence)` দিয়ে, ফলে বাইনারি ফাইলের k-তম রেকর্ড আগের $k-1$টি না পড়েই পড়া যায়।



তিনটি শুরুবিন্দু, একটি অফসেট — এই-ই পুরো র‍্যান্ডম অ্যাক্সেস

কল	ফল · রিটার্ন
fseek(fp, off, whence)	শুরুবিন্দু + off-এ যায় · সফল হলে 0
ftell(fp)	বর্তমান অফসেট, long হিসেবে · এররে -1L
rewind(fp)	0-তে ফেরে **এবং** এরর ফ্ল্যাগ মোছে · void
feof(fp)	কোনো পড়া শেষে **ইতিমধ্যে ঠেকলে** অশূন্য
ferror(fp) / clearerr(fp)	এরর ফ্ল্যাগ আছে? / দুটি ফ্ল্যাগই মোছে
fflush(fp)	আউটপুট বাফার এখনই লিখে দেয়
remove(n) / rename(o, n)	ফাইল মোছে / নাম বদলায় · সফল হলে 0

সমাধানকৃত উদাহরণ ✍️ ছাত্রদের রেকর্ডের বাইনারি ফাইল: `struct student { int roll; char name[16]; float cgpa; };` — 4 + 16 + 4 = **24 বাইট** (এখানে প্যাডিং লাগে না; প্যাডিংয়ের নিয়ম স্ট্রাকচার লেসনে)। **(ক)** ফাইলের আকার: `fseek(fp, 0, SEEK_END); long n = ftell(fp);` → n = 240 ⇒ 240 ÷ 24 = **১০টি রেকর্ড**। **(খ) ৫ম** রেকর্ড পড়া (k = 4): `fseek(fp, 4 * sizeof(struct student), SEEK_SET)` → বাইট **96**, তারপর `fread(&s, sizeof s, 1, fp)` ফেরত দেয় 1। **(গ)** এখন `ftell(fp)` = 96 + 24 = **120**। **(ঘ)** জায়গাতেই হালনাগাদ: **"rb+"** দিয়ে খোলো, `fseek(fp, -24L, SEEK_CUR)`, তারপর `fwrite(&s, sizeof s, 1, fp)`। **(ঙ)** শেষ রেকর্ড: `fseek(fp, -24L, SEEK_END)` → বাইট 216।

```
/* WRONG: the last line prints twice */
while (!feof(fp)) {
    fgets(line, 80, fp);
    fputs(line, stdout);
}
/* RIGHT: test the read itself */
while (fgets(line, sizeof line, fp) != NULL)
    fputs(line, stdout);
```

EOF একটি মান, বাইট নয়: EOF হলো <stdio.h>-এর একটি ম্যাক্রো, একটি **ঋণাত্মক int** (সাধারণত -1), যা কিছু না থাকলে ইনপুট ফাংশন ফেরত দেয়। ফাইলে কোনো EOF অক্ষর জমা থাকে না; ফাইল শুধু শেষ হয়। তাই **fgetc ফেরত দেয় int**: এতে 256টি বাইট-মান *এবং* EOF — সবই আঁটতে হবে।

পরীক্ষার ফাঁদ: (১) **feof() সামনে দেখে না**: কোনো পড়া ব্যর্থ হওয়ার পরেই এটি সত্য হয়, তাই `while (!feof(fp))` লুপ একবার বেশি চলে। (২) ftell দিয়ে বাইটের হিসাবের জন্য **বাইনারি** মোড লাগে। (৩) লেখা আর পড়ার মাঝে fseek বা fflush ডাকো।

মনে রাখার কৌশল: "**SET, CUR, END = 0, 1, 2 — শুরু, এখানে, শেষ।**" আর "**আগে পড়ো, পরে জিজ্ঞেস করো**": fgetc/fgets/fread-এর ফেরত মান যাচাই করো; feof() জিজ্ঞেস করো কেবল *কেন* থামল জানতে।

৫ পরিবর্তনশীল দৈর্ঘ্যের আর্গুমেন্ট তালিকা (stdarg.h)

সংজ্ঞা: **ভেরিয়াডিক ফাংশন** (variadic function) পরিবর্তনশীল সংখ্যক আর্গুমেন্ট নেয়, যা ঘোষণা করা হয় **অন্তত একটি নামযুক্ত প্যারামিটারের** পরে একটি এলিপসিস `...` দিয়ে: `int printf(const char *fmt, ...);`। `<stdarg.h>`-এর ম্যাক্রোগুলো নামহীন আর্গুমেন্ট একটা একটা করে পড়ে; কতগুলো এল তা ফাংশনের নিজের **জানার কোনো উপায় নেই**।

```
#include <stdarg.h>
int sum(int count, ...) {
    va_list ap;
    int total = 0;
    va_start(ap, count);
    for (int i = 0; i < count; i++)
        total += va_arg(ap, int);
    va_end(ap);
    return total;
}
sum(4, 10, 20, 30, 40)   → 100
sum(2, 5, 15)            → 20
```

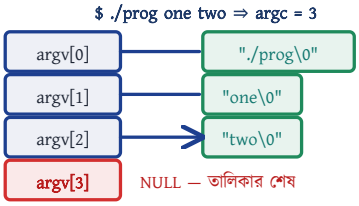
ম্যাক্রো / টাইপ	কাজ
va_list ap	আর্গুমেন্টগুলোর ওপর কার্সর
va_start(ap, last)	**শেষ নামযুক্ত** প্যারামিটারের পর থেকে শুরু
va_arg(ap, T)	পরেরটি T টাইপ হিসেবে আনে
va_copy(d, s)	কার্সরের কপি (C99)
va_end(ap)	পরিষ্কার করে — প্রতি va_start-এ একটি

পরীক্ষার ফাঁদ: নামহীন আর্গুমেন্টে **ডিফল্ট প্রোমোশন** হয়: char, short → **int**; float → **double**। তাই **va_arg(ap, float)** ও va_arg(ap, char) অনির্ধারিত — double / int চাও। শেষ আর্গুমেন্টের পরে পড়াও অনির্ধারিত।

কলি সংখ্যা জানে কীভাবে: একটি **count** আর্গুমেন্ট (ওপরের sum) · শেষে একটি **সেন্টিনেল** (NULL, -1) · একটি **ফরম্যাট স্ট্রিং** — printf % চিহ্ন গোনে। সংখ্যা ভুল হলে va_arg আবর্জনা পড়ে।

৬ কমান্ড-লাইন প্যারামিটার (argc, argv)

সংজ্ঞা: `int main(int argc, char *argv[])` (বা `char **argv`) কমান্ডের পরে টাইপ করা শব্দগুলো পায়। **argc** = কতগুলো শব্দ, **প্রোগ্রামের নামসহ**; **argv[0]** = প্রোগ্রামের নাম; argv[1] … argv[argc − 1] = আর্গুমেন্টগুলো, **সবসময় স্ট্রিং হিসেবে**; এবং **argv[argc] = NULL**।



argv হলো স্ট্রিং-পয়েন্টারের একটি অ্যারে

সমাধানকৃত উদাহরণ ✍️ `$ ./sum 10 20 30`: argc = 4; argv[1] হলো **স্ট্রিং** "10", তাই যোগফলের জন্য রূপান্তর লাগে: `for (i = 1; i < argc; i++) s += atoi(argv[i]);` → 60। **(খ)** `$ ./prog "hello world"` → argc = 2: শেলের উদ্ধৃতিচিহ্ন একে একটি আর্গুমেন্ট বানায়। **(গ)** শুধু `$ ./prog` → argc = 1, argv[1] = NULL: আগে `argc < 2` যাচাই করো। **(ঘ)** atoi("12ab") চুপচাপ 12 দেয় — **strtol** এরর জানাতে পারে।

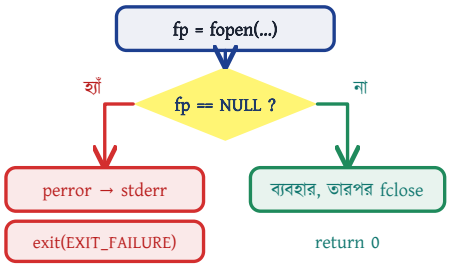
পরীক্ষার ফাঁদ: (১) বাস্তবে argc কখনো 0 নয় — নামটাও গোনা হয়। (২) `argv[1] + argv[2]` যোগ করে **পয়েন্টার**, সংখ্যা নয়। (৩) argc/argv নাম শুধু প্রথা; স্ট্যান্ডার্ড যা ঠিক করে দেয় তা হলো **টাইপ**।

মনে রাখার কৌশল: "**argc নামও গোনে; argv শেষ হয় NULL-এ।**" stdarg-এর জন্য: "**start, arg, arg, …, end**" — আর "**ছোটরা বড় হয়**": char → int, float → double।

৭ এরর হ্যান্ডলিং: রিটার্ন মান, errno, perror, assert, exit

সংজ্ঞা: C-তে **এক্সেপশন (exception) নেই**। লাইব্রেরি ফাংশন ব্যর্থতা জানায় তার **রিটার্ন মান** দিয়ে এবং অনেকে errno-তে একটি কোড রেখে; প্রোগ্রামকে **যাচাই** করতে হয়, stderr-এ জানাতে হয় এবং **ব্যর্থতার এক্সিট স্ট্যাটাস** দিয়ে শেষ করতে হয়। যাচাই বাদ দিলে আপনা-আপনি কিছুই ঘটে না।

কল	ব্যর্থতার চেহারা
fopen, malloc, calloc, fgets	NULL
scanf, fscanf	চাওয়ার চেয়ে কম আইটেম, বা EOF
fgetc, getchar, fclose, fputs	EOF
printf, fprintf	একটি **ঋণাত্মক** মান
fseek / ftell	অশূন্য / -1L
fread, fwrite	n-এর চেয়ে কম আইটেম
sqrt(-1) · strtol ওভারফ্লো	errno = **EDOM** · **ERANGE**



যাচাই করো, জানাও, বেরিয়ে যাও — প্রতিটি লাইব্রেরি কলে

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    perror("data.txt");
    fprintf(stderr, "code %d: %s\n",
            errno, strerror(errno));
    exit(EXIT_FAILURE);
}
```

errno · perror · strerror: **errno** (<errno.h>) ব্যর্থ কল একটি ধনাত্মক কোডে বসায়, আর লাইব্রেরি একে **কখনো 0-তে ফেরায় না**। ISO C সংজ্ঞায়িত করে কেবল **EDOM, ERANGE, EILSEQ**; ENOENT ইত্যাদি POSIX-এর। **perror("data.txt")** stderr-এ ছাপে **data.txt: No such file or directory**; **strerror(errno)** (<string.h>) সেই লেখাটি স্ট্রিং হিসেবে দেয়।

assert, exit, abort — প্রোগ্রাম যেভাবে শেষ হয়:

assert(expr): মিথ্যা হলে stderr-এ এক্সপ্রেশন, ফাইল, লাইন ছাপে, তারপর **abort()**

<assert.h>-এর আগে **#define NDEBUG** ⇒ সব assert **উধাও**

exit(status): স্ট্রিম ফ্লাশ ও বন্ধ করে, atexit() হ্যান্ডলার চালায়

main-এ **return n;** ≡ **exit(n)** · EXIT_SUCCESS 0 · EXIT_FAILURE ≠ 0

abort(): অস্বাভাবিক সমাপ্তি, atexit হ্যান্ডলার চলে না। শেল স্ট্যাটাস পড়ে **$?** / **%ERRORLEVEL%** দিয়ে।

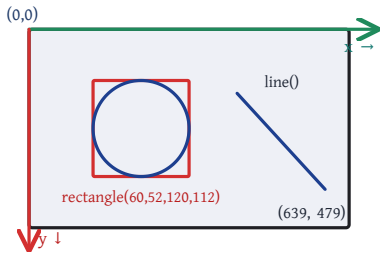
সমাধানকৃত উদাহরণ ✍ শূন্য দিয়ে ভাগ: b = 0 হলে পূর্ণসংখ্যার `a / b` **অনির্ধারিত আচরণ** — কোনো রিটার্ন কোড নেই, errno নেই, সাধারণত ক্র্যাশ — তাই **আগে যাচাই**: `if (b == 0) { fputs("divide by zero\n", stderr); return EXIT_FAILURE; }`। (IEEE 754 float-এ 1.0 / 0.0 = **inf**।)

পরীক্ষার ফাঁদ: (১) errno অর্থবহ কেবল ব্যর্থ কলের **ঠিক পরে**। (২) **assert** ইনপুট যাচাই নয় — রিলিজ বিল্ডে উধাও হয়। (৩) এরর জানাও **stderr**-এ, stdout-এ নয়। (৪) **0 = সাফল্য**, অশূন্য = ব্যর্থতা — উল্টো নয়।

মনে রাখার কৌশল: **"যাচাই, জানাও, বেরোও।"** রিটার্ন মান যাচাই করো, stderr-কে জানাও (perror), EXIT_FAILURE দিয়ে বেরোও। **"বাগের জন্য assert, ব্যবহারকারীর জন্য if।"**

৮ C-তে গ্রাফিক্স (graphics.h, Turbo C)

সংজ্ঞা: **graphics.h** হলো DOS-এর Turbo C/C++-এর জন্য Borland-এর **BGI** (Borland Graphics Interface) — **ISO C-র অংশ নয়** (gcc-তে এমন হেডার নেই)। স্ক্রিন হলো পিক্সেলের গ্রিড, যার **মূলবিন্দু (0, 0) ওপরে-বাঁয়ে** এবং **y বাড়ে নিচের দিকে**; VGA হাই মোডে (640 × 480, 16 রং) `getmaxx()` = 639 ও `getmaxy()` = 479।



স্ক্রিন স্থানাঙ্ক: y নির্দেশ করে **নিচে**

ফাংশন	কাজ
initgraph(&gd, &gm, path)	গ্রাফিক্স মোড; gd = DETECT
graphresult() · closegraph()	grOk (0) · টেক্সটে ফেরা
putpixel(x,y,c) · line(x1,y1,x2,y2)	একটি পিক্সেল · একটি রেখা
circle(x,y,r) · rectangle(l,t,r,b)	ব্যাসার্ধ · **দুই কোণ**
setcolor(c) · setbkcolor(c)	BLACK 0 · RED 4 · WHITE 15
setfillstyle + floodfill(x,y,b)	সীমানা b পর্যন্ত ভরাট
outtextxy(x,y,s) · cleardevice()	লেখা · স্ক্রিন মোছা

```c
#include <graphics.h>
#include <conio.h>    /* getch() */
int main(void) {
  int gd = DETECT, gm;
  initgraph(&gd, &gm, "C:\\TC\\BGI");
  if (graphresult() != grOk) return 1;
  setcolor(RED);  rectangle(100,100,200,200);
  setcolor(BLUE); circle(150, 150, 50);
  outtextxy(110, 220, "Graphics Example");
  getch(); closegraph(); return 0;
}
```

পরীক্ষার ফাঁদ: (১) **getch()** আছে **conio.h**-এ, graphics.h-এ নয় — এটি শুধু একটি কী চাপা পর্যন্ত ছবিটা ধরে রাখে। (২) rectangle নেয় **দুই কোণ**, প্রস্থ-উচ্চতা নয়। (৩) বড় y মানে **নিচে**।

৯ স্ট্যান্ডার্ড লাইব্রেরি ফাংশন ও হেডার

সংজ্ঞা: **C স্ট্যান্ডার্ড লাইব্রেরি** হলো কম্পাইলারের সঙ্গে আসা আগে-কম্পাইল-করা কোড। **হেডার ঘোষণা করে** (প্রোটোটাইপ, ম্যাক্রো, টাইপ); **লাইব্রেরি আর্কাইভ সংজ্ঞা দেয়** (যে মেশিন কোড লিংকার যোগ করে)। C89-এ স্ট্যান্ডার্ড হেডার **১৫টি**, C99-এ **২৪টি**, C11-এ **২৯টি**।

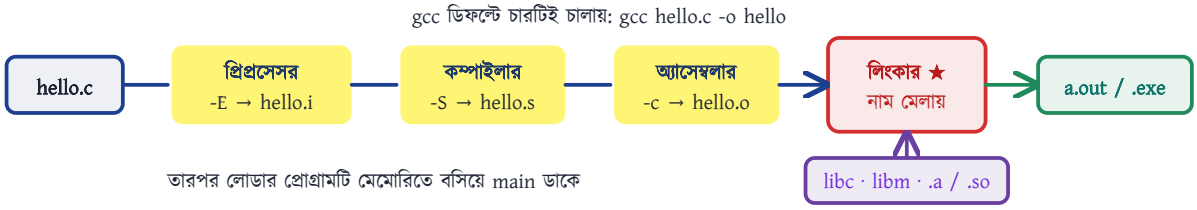
হেডার	হাই-ইল্ড সদস্য
<stdio.h>	printf, scanf, fopen, fgets, fread, perror, EOF, FILE, NULL
<stdlib.h>	malloc, calloc, realloc, free, atoi, strtol, **rand, srand, exit, abs**, qsort, system
<string.h>	strlen, strcpy, strcat, strcmp, strchr, strstr, memcpy, memset, strerror
<math.h>	sqrt, pow, fabs, ceil, floor, sin, cos, log, log10, exp — সবই ফেরত দেয় **double**
<ctype.h> · **<time.h>**	isalpha, isdigit, toupper, tolower · time, clock, difftime
<limits.h> · **<stddef.h>**	INT_MAX, CHAR_BIT · size_t, NULL, offsetof

সমাধানকৃত উদাহরণ ✍ আউটপুট: strlen("Dhaka") = **5**, sizeof("Dhaka") = **6** · pow(2, 10) = **1024.000000** · floor(-2.1) = **-3.0** · `srand(time(NULL))`-এর পর `rand() % 6 + 1` = একটি ছক্কার দান।

মনে রাখার কৌশল: "**abs থাকে stdlib-এ, fabs থাকে math-এ।**" "**হেডার ঘোষণা করে, লাইব্রেরি সংজ্ঞা দেয়**" — কোড আনে শুধু লিংকার।

১০ সোর্স থেকে এক্সিকিউটেবল: কম্পাইলিং ও লিংকিং

সংজ্ঞা: একটি C প্রোগ্রাম তৈরি হয় চার ধাপে, আর **লিংকার (linker)** শেষ ধাপ: এটি অবজেক্ট ফাইল ও লাইব্রেরি জুড়ে একটি এক্সিকিউটেবল বানায় **সিম্বল রিজলভ করে** — প্রতিটি বাইরের নামের ব্যবহারকে ঠিক একটি সংজ্ঞার সঙ্গে মিলিয়ে — এবং ঠিকানা **রিলোকেট** করে। এরপর **লোডার** এক্সিকিউটেবলটি মেমোরিতে বসিয়ে চালু করে।



যেকোনো ধাপের পরে থামো তার ফ্ল্যাগ দিয়ে: -E, -S বা -c

ধাপ	ইন → আউট	কী করে · সাধারণ এরর
প্রিপ্রসেসর	.c → .i	#include, #define, #if; কমেন্ট বাদ দেয় (প্রিপ্রসেসর লেসন) · হেডার ফাইল না পাওয়া
কম্পাইলার	.i → .s	সিনট্যাক্স ও টাইপ যাচাই, অ্যাসেম্বলি কোড · সেমিকোলন বাদ, টাইপ অমিল
অ্যাসেম্বলার	.s → .o / .obj	**অমীমাংসিত** বাইরের সিম্বলসহ মেশিন কোড
লিংকার ★	.o + libs → exe	রিজলভ ও রিলোকেট করে · **undefined reference**, multiple definition

দিক	স্ট্যাটিক লিংকিং	ডাইনামিক (শেয়ার্ড) লিংকিং
লাইব্রেরি ফাইল	.a (Linux) · .lib (Windows)	.so (Linux) · .dll (Windows)
কখন যুক্ত	লিংকের সময়: **কেবল দরকারি অবজেক্ট মডিউল** কপি হয়	লোডের সময় ডাইনামিক লিংকার দিয়ে (বা রানটাইমে: dlopen, LoadLibrary)
আকার · মেমোরি	বড় ফাইল; প্রতিটি প্রোগ্রামের নিজস্ব কপি	ছোট ফাইল; মেমোরিতে **একটি কপি ভাগাভাগি**
লাইব্রেরি হালনাগাদ	প্রতিটি প্রোগ্রাম আবার লিংক	.so/.dll একবার বদলালেই হয়
ঝুঁকি	রানটাইমে নেই — স্বয়ংসম্পূর্ণ	লাইব্রেরি না থাকা বা ভুল সংস্করণ ("DLL hell")

```
/* util.h */ extern int counter;
              int add(int a, int b);
/* util.c */ int counter = 0;
              int add(int a, int b)
              { counter++; return a + b; }
/* main.c */ #include "util.h"
gcc -c main.c      → main.o
gcc -c util.c      → util.o
gcc main.o util.o -o prog
gcc prog.c -o prog -lm   (libm)
```

extern ও লিংকেজ: extern এমন একটি নাম ঘোষণা করে যার সংজ্ঞা **অন্য কোথাও**; সংজ্ঞা থাকতে হবে ঠিক **একটি** ফাইলে। ফাইল স্কোপে static দেয় **ইন্টারনাল লিংকেজ**: নামটি লিংকার ও অন্য ফাইলের কাছে অদৃশ্য (স্টোরেজ ক্লাস লেসন)। -lm গণিত লাইব্রেরি libm যুক্ত করে; একে সোর্স ফাইলের **পরে** রাখো।

পরীক্ষার ফাঁদ: (১) <math.h> দেয় কেবল প্রোটোটাইপ; "**undefined reference to sqrt**" একটি **লিংকার** এরর, সারে -lm দিয়ে। (২) স্ট্যাটিক লিংকিং পুরো লাইব্রেরি কপি করে **না**।

মনে রাখার কৌশল: "Please Compile And Link" — Preprocess, Compile, Assemble, Link; ফ্ল্যাগ -E -S -c প্রথম তিনটির পরে থামায়। "**স্ট্যাটিক = সঙ্গে স্যুটকেস, ডাইনামিক = পৌঁছে ধার।**"

পরীক্ষার আগের রাতের পাতা

তথ্য-পত্র:

stdin 0 · stdout 1 · stderr 2

stderr **আনবাফারড**

%d int · %u unsigned · %ld long

%c char · %s স্ট্রিং · %p void *

%x হেক্স · %o অক্টাল · %e সূচক

size_t → %zu

printf-এ %f = float ও double

scanf: %f float, %lf double

% ফ্ল্যাগ প্রস্থ .প্রিসিশন দৈর্ঘ্য অক্ষর

প্রস্থ ন্যূনতম, কখনো কাটে না

printf ফেরত দেয় ছাপা অক্ষর

scanf ফেরত দেয় আইটেম সংখ্যা / EOF

scanf-এ & লাগে, অ্যারেতে নয়

gets C11-এ বাতিল → fgets

fgets '\n' রাখে, পড়ে n - 1

getchar ফেরত দেয় **int**

fopen ব্যর্থ ⇒ **NULL**

"w" কেটে ফেলে · "a" জোড়ে

"r", "r+" কখনো ফাইল বানায় না

b = বাইনারি · x = এক্সক্লুসিভ (C11)

fread/fwrite ফেরত দেয় সম্পন্ন আইটেম

SEEK_SET 0 · CUR 1 · END 2

আকার = fseek END + ftell

rewind = 0-তে seek + এরর মোছা

feof সত্য কেবল ব্যর্থ পড়ার **পরে**

EOF = ঋণাত্মক int, সাধারণত -1

va_start · va_arg · va_end

float → double, char → int

argc প্রোগ্রামের নামও গোনে

argv[argc] = NULL

perror → stderr

errno: EDOM · ERANGE · EILSEQ

NDEBUG দিলে assert বন্ধ

EXIT_SUCCESS 0 · EXIT_FAILURE

getch() আছে conio.h-এ

BGI: (0,0) ওপরে-বাঁয়ে, y নিচে

-E · -S · -c · লিংক

স্ট্যাটিক .a/.lib · ডাইনামিক .so/.dll

-lm গণিত লাইব্রেরি যুক্ত করে

abs stdlib-এ · fabs math-এ

MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

১. printf ও scanf ঘোষিত **<stdio.h>**-এ
২. `printf("%d", printf("Hello"))` ⇒ **Hello5**
৩. 3.14159-এর `"%8.3f"` ⇒ **___3.142**
৪. double পড়তে: **scanf("%lf", &d)**
৫. "r"-এ নেই এমন ফাইল fopen ⇒ **NULL**
৬. 24-বাইট রেকর্ডের ৫ম রেকর্ড শুরু বাইট **96**-এ
৭. `./prog one two` ⇒ argc = **3**
৮. ভেরিয়াডিক ম্যাক্রো থাকে **<stdarg.h>**-এ
৯. perror("x") ⇒ "x: <কারণ>" **stderr**-এ
১০. assert মিথ্যা ⇒ বার্তা + **abort()**
১১. undefined reference to sqrt ⇒ **-lm** যোগ
১২. শেয়ার্ড লাইব্রেরি যুক্ত হয় **লোডের সময়**

সবচেয়ে বড় ফাঁদগুলো: scanf-এ double-এর জন্য %f · size_t-এর জন্য %lu বা %d · scanf-এ & ভুলে যাওয়া · `char c = getchar()` · **while (!feof(fp))** · ডেটা রাখার দরকার থাকলেও "w" · fopen-কে EOF-এর সঙ্গে মেলানো · va_arg(ap, float) · argc প্রোগ্রামের নাম বাদ দেয় বলা · stdout-এ এরর · ইনপুট যাচাইয়ে assert · getch()-কে graphics.h-এ রাখা · **স্ট্যাটিক লিংকিং পুরো লাইব্রেরি কপি করে বলা**।

সব কৌশল একসাথে: "ফ্ল্যাগ, প্রস্থ, ডট, দৈর্ঘ্য, অক্ষর" · "printf নেয় মান, scanf নেয় ঠিকানা" · "lf = long float = double" · "r পড়ে, w মোছে, a জোড়ে; + দেয় বাকি অর্ধেক; b মানে বাইট" · "SET, CUR, END = 0, 1, 2" · "আগে পড়ো, পরে জিজ্ঞেস" · "start, arg, arg, ..., end; ছোটরা বড় হয়" · "argc নামও গোনে; argv শেষ হয় NULL-এ" · "যাচাই, জানাও, বেরোও" · "বাগের জন্য assert, ব্যবহারকারীর জন্য if" · "abs stdlib-এ, fabs math-এ" · "Please Compile And Link" · "স্ট্যাটিক = স্যুটকেস, ডাইনামিক = পৌঁছে ধার"।

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

১. scanf দিয়ে একটি **double** পড়তে কোন কনভার্শন লাগে?
ক) %f খ) %d
গ) %lf ঘ) %Lf

২. ফাইল খুলতে না পারলে fopen() কী ফেরত দেয়?
ক) একটি NULL পয়েন্টার খ) EOF
গ) -1 ঘ) একটি খালি FILE

৩. `int main(int argc, char *argv[])` সহ প্রোগ্রাম `./prog one two` হিসেবে চালানো হলো। argc কত?
ক) 1 খ) 2
গ) 4 ঘ) 3

৪. কোন মোড লেখার জন্য ফাইল খোলে এবং আগের সব লেখা **মুছে ফেলে**?
ক) "a" খ) "w"
গ) "r+" ঘ) "a+"

৫. `printf("%5.2f|", 3.14159);` ছাপে (_ = একটি স্পেস):
ক) 3.14| খ) _3.14|
গ) 3.14159| ঘ) 00003.14|

৬. va_start, va_arg ও va_end সংজ্ঞায়িত কোথায়?
ক) <stdio.h> খ) <stdlib.h>
গ) <stdarg.h> ঘ) <conio.h>

৭. gcc জানাল "undefined reference to sqrt"। সমাধান হলো:
ক) লিংকের সময় -lm যোগ খ) #include <math.h> যোগ
গ) -c দিয়ে কম্পাইল ঘ) -E দিয়ে কম্পাইল

৮. **feof(fp)** সম্পর্কে কোনটি সত্য?
ক) পরের পড়া ব্যর্থ হবে আগেই খ) EOF অক্ষর ফেরত দেয় বলে
গ) conio.h-এ ঘোষিত ঘ) কোনো পড়া শেষে ঠেকার পরেই সত্য হয়

৯. perror() তার বার্তা লেখে কোথায়?
ক) stdout খ) একটি লগ ফাইলে
গ) stderr ঘ) stdin

১০. **স্ট্যাটিক** লিংকিংয়ে লিংকার:
ক) রানটাইমে লাইব্রেরি লোড করে খ) কেবল দরকারি অবজেক্ট মডিউল কপি করে
গ) এক্সিকিউটেবল ছোট করে ঘ) রানটাইমে .dll চায়

উত্তরমালা → ১-গ · ২-ক · ৩-ঘ · ৪-খ · ৫-খ · ৬-গ · ৭-ক · ৮-ঘ · ৯-গ · ১০-খ

লিখিত প্রশ্ন: একটি C প্রোগ্রাম লেখো যা কমান্ড লাইন থেকে একটি ফাইলের নাম নেয়, ফাইলের পূর্ণসংখ্যাগুলো পড়ে তাদের যোগফল ছাপে এবং প্রতিটি এরর ঠিকভাবে সামলায়। argc/argv এবং তোমার প্রতিটি যাচাই ব্যাখ্যা করো। ১০

১ **আর্গুমেন্ট** — `int main(int argc, char *argv[]);` argc প্রোগ্রামের নামও গোনে, তাই একটি ফাইলের নাম মানে **argc == 2**, আর argv[1] হলো নামটি। `argc != 2` হলে stderr-এ ব্যবহারবিধি ছাপো এবং `return EXIT_FAILURE;`। ২

২ **খোলা** — `FILE *fp = fopen(argv[1], "r");` এবং `if (fp == NULL) { perror(argv[1]); return EXIT_FAILURE; }` — perror errno থেকে কারণটি যোগ করে, stderr-এ। ২

৩ **পড়া** — `long sum = 0; int x; while (fscanf(fp, "%d", &x) == 1) sum += x;` — লুপটি **রিটার্ন মান** যাচাই করে (feof নয়), তাই ফাইল শেষে ও ভুল ডেটায় থামে; তারপর `if (ferror(fp))` হলে পড়ার এরর জানাও। ৩

৪ **শেষ** — `fclose(fp); printf("Sum = %ld\n", sum); return EXIT_SUCCESS;` — স্ট্যাটাস 0 শেলকে জানায় কাজ সফল। ১

৫ **ব্যাখ্যা** — argv রাখে স্ট্রিং, argv[argc] হলো NULL; C-তে এক্সেপশন নেই বলে প্রতিটি লাইব্রেরি কল যাচাই করা হয়েছে: fopen-এর NULL, fscanf-এর আইটেম সংখ্যা, ferror-এর ফ্ল্যাগ। ২

পরীক্ষার আগে ✓

- ✓ যেকোনো প্রস্থ, প্রিসিশন ও ফ্ল্যাগে printf-এর আউটপুট বলতে পারি
- ✓ ছয়টি fopen মোড এবং আগের ফাইলে প্রত্যেকটি কী করে বলতে পারি
- ✓ রেকর্ড k-এর জন্য fseek অফসেট বের করতে পারি
- ✓ চারটি বিল্ড ধাপের নাম বলতে ও স্ট্যাটিক-ডাইনামিক লিংকিংয়ের তুলনা করতে পারি